Einführung in die Rechnerarchitektur

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Überblick der nächsten Wochen

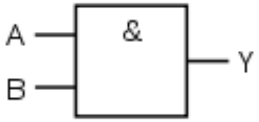
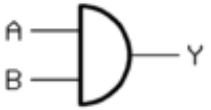
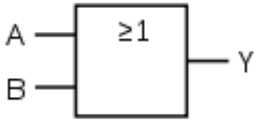
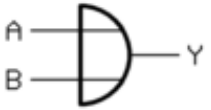
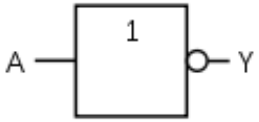
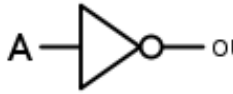
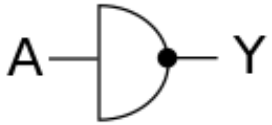
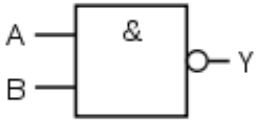
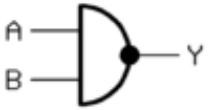
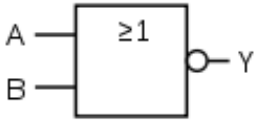
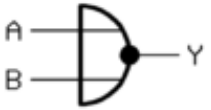
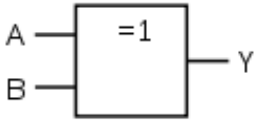
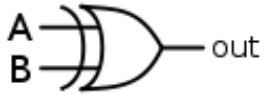
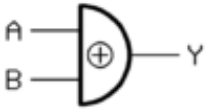
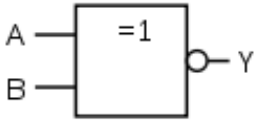
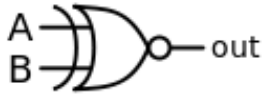
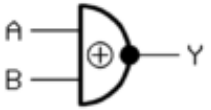
=

- **Grundlagen**
  - Beschreibungen über „0“ und „1“ (Boolesche Algebra)
  - Beschreibungen von Schaltungen
- **Rechnen**
  - Digitale Schaltungen für Addition, Subtraktion, Multiplikation
- **Speichern & Steuern**
  - Sequentielle Schaltungen
  - Speicherelemente
  - Endliche Automaten
- **Prozessor**
  - Aufbau
  - Optimierungen
- **Entwerfen**
  - Synthese von allgemeinen Schaltungen
  - Logikminimierung

Schaltkreis

- Gerichteter, zyklenfreier Graph
- Knoten repräsentieren
  - Primäre Eingänge
  - Primäre Ausgänge
  - Gatter (i.d.R. basierend auf vorher festgelegter Gatterbibliothek)
- Kanten repräsentieren
  - Signale zwischen den Gattern bzw. primären Eingängen/Ausgängen
- Gängige Kostenmaße
  - Anzahl der Gatter (Größe)
  - Tiefe, d.h. Zahl der Gatter auf dem längsten Pfad von einem primären Eingang zu einem primären Ausgang (Geschwindigkeit)

DIN/IEC-Symbole

Name	Funktion	Symbol in Schaltplan		
		IEC 60617-12 : 1997 & ANSI/IEEE Std 91/91a-1991	ANSI/IEEE Std 91/91a-1991	DIN 40700 (vor 1976)
Und-Gatter (AND)	$Y = A \wedge B$ $Y = A \cdot B$			
Oder-Gatter (OR)	$Y = A \vee B$ $Y = A + B$			
Nicht-Gatter (NOT)	$Y = \overline{A}$ $Y = \neg A$			
NAND-Gatter (NICHT UND) (NOT AND)	$Y = \overline{A \wedge B}$ $Y = A \overline{\wedge} B$			
NOR-Gatter (NICHT ODER) (NOT OR)	$Y = \overline{A \vee B}$ $Y = A \overline{\vee} B$			
XOR-Gatter (Exklusiv-ODER, Antivalenz) (EXCLUSIVE OR)	$Y = A \underline{\vee} B$ $Y = A \oplus B$			
XNOR-Gatter (Exklusiv-Nicht-ODER, Äquivalenz) (EXCLUSIVE NOT OR)	$Y = \overline{A \underline{\vee} B}$ $Y = \overline{A \oplus B}$			

Multiplikation

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Multiplizierer

- Multiplikation zweier Bits
  $B_0 * A_0 = B_0 A_0$
  $\Rightarrow$ Und-Verknüpfung

- Multiplikation einer mehrstelligen Zahl mit einem Bit
  $(B_1,B_0) * A_0 = (B_1A_0 , B_0A_0)$
  $\Rightarrow$ Und-Verknüpfung der Stellen mit dem Bit

Multiplizierer

- Multiplikation zweier mehrstelliger Zahlen
  $(B_1,B_0) * (A_1,A_0)$
  $= (B_1,B_0) * A_0 + (B_1,B_0)*A_1*2$
  $= (B_1A_0 , B_0A_0) + (B_1A_1 , B_0A_1 ,0)$

```
1 0 1 1 * 0 1 0 1        11*5
---------------------
0 0 0 0
  1 0 1 1
    0 0 0 0
      1 0 1 1
    1
---------------------
0 1 1 0 1 1 1            55
```

  - Berechnung der Teilprodukte $(B_{n-1},...B_0)*A_i$, mit $0 \leq i < n$
  - Stellengerechte Summenbildung
  - Teilprodukt mit A_i muss um i Stellen nach links verschoben werden

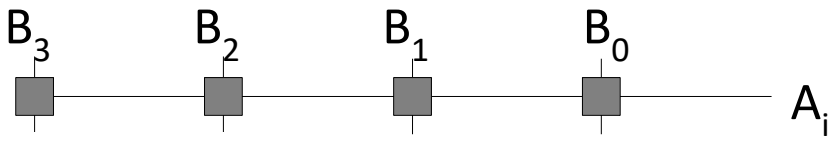
Multiplizierer

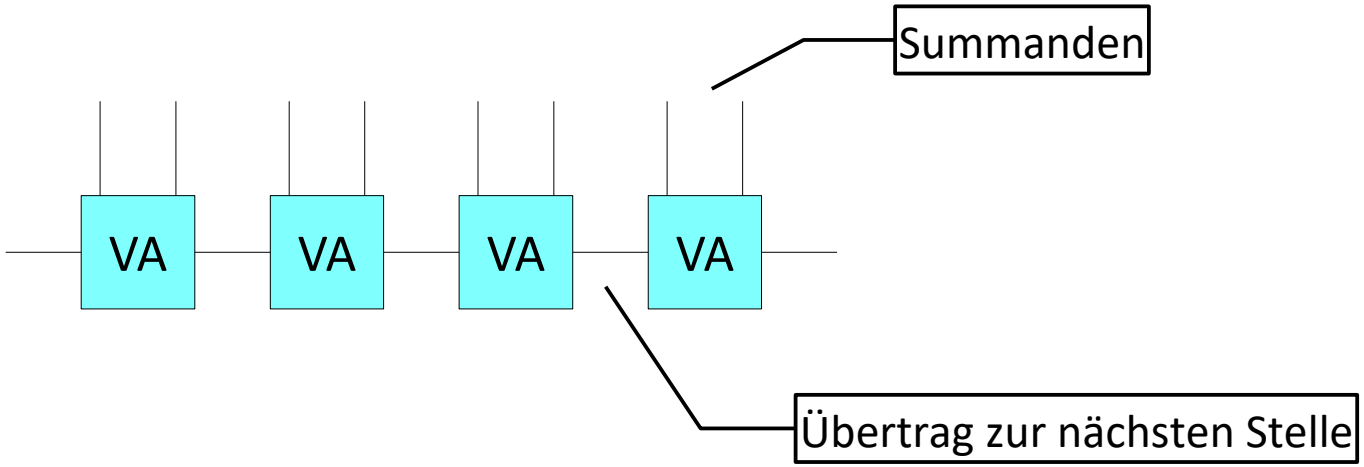
- Verschiedene Varianten möglich
  - Vollständig parallele Schaltung
  - Bildung und Aufsummieren der Teilprodukte nacheinander
  - Mischformen

Vereinfachte Darstellung

- Und-Verknüpfung von *B* mit A_i

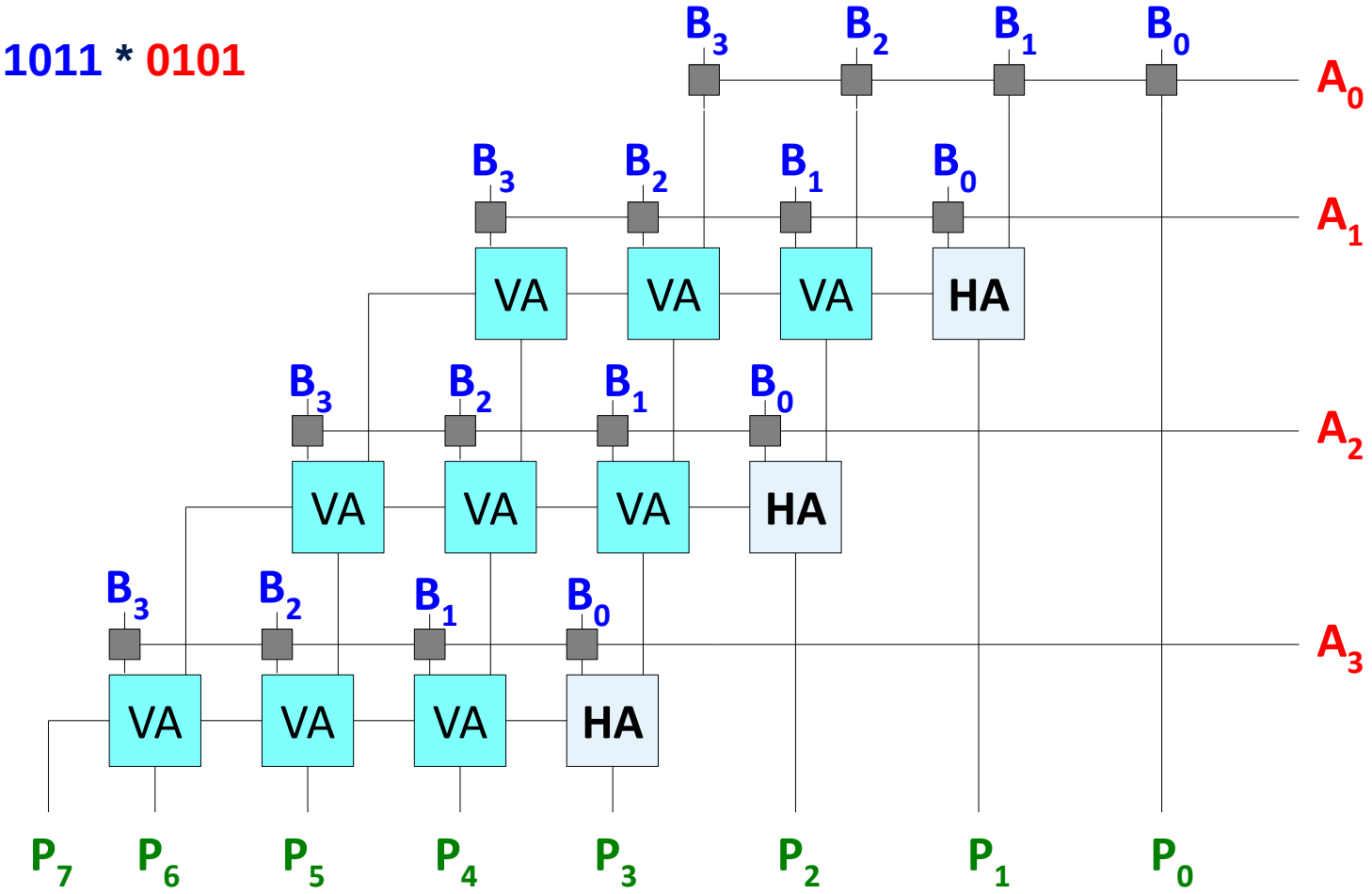
- Volladdierer-Kette





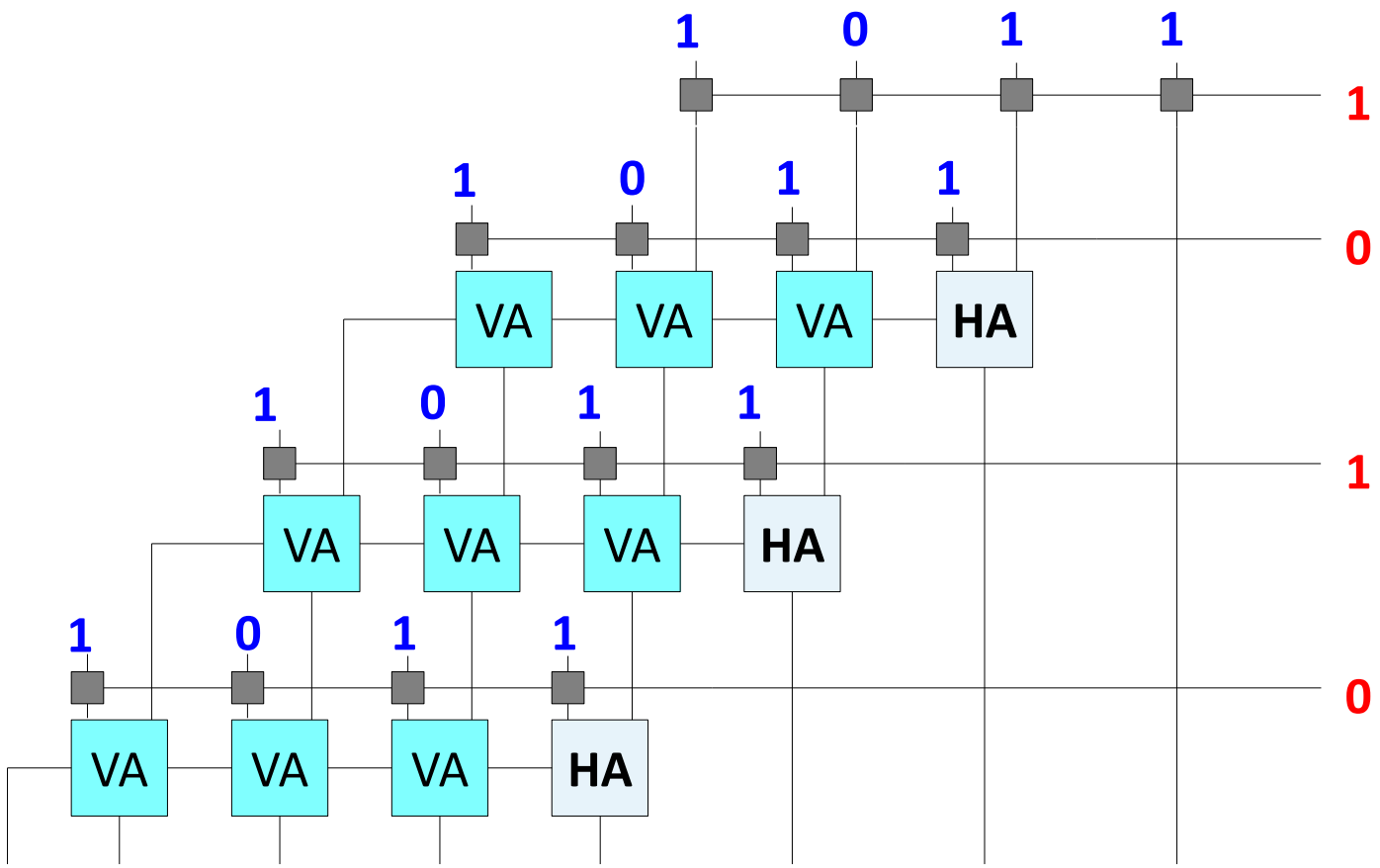
Paralleler Multiplizierer #1

Beispiel: 1011 * 0101



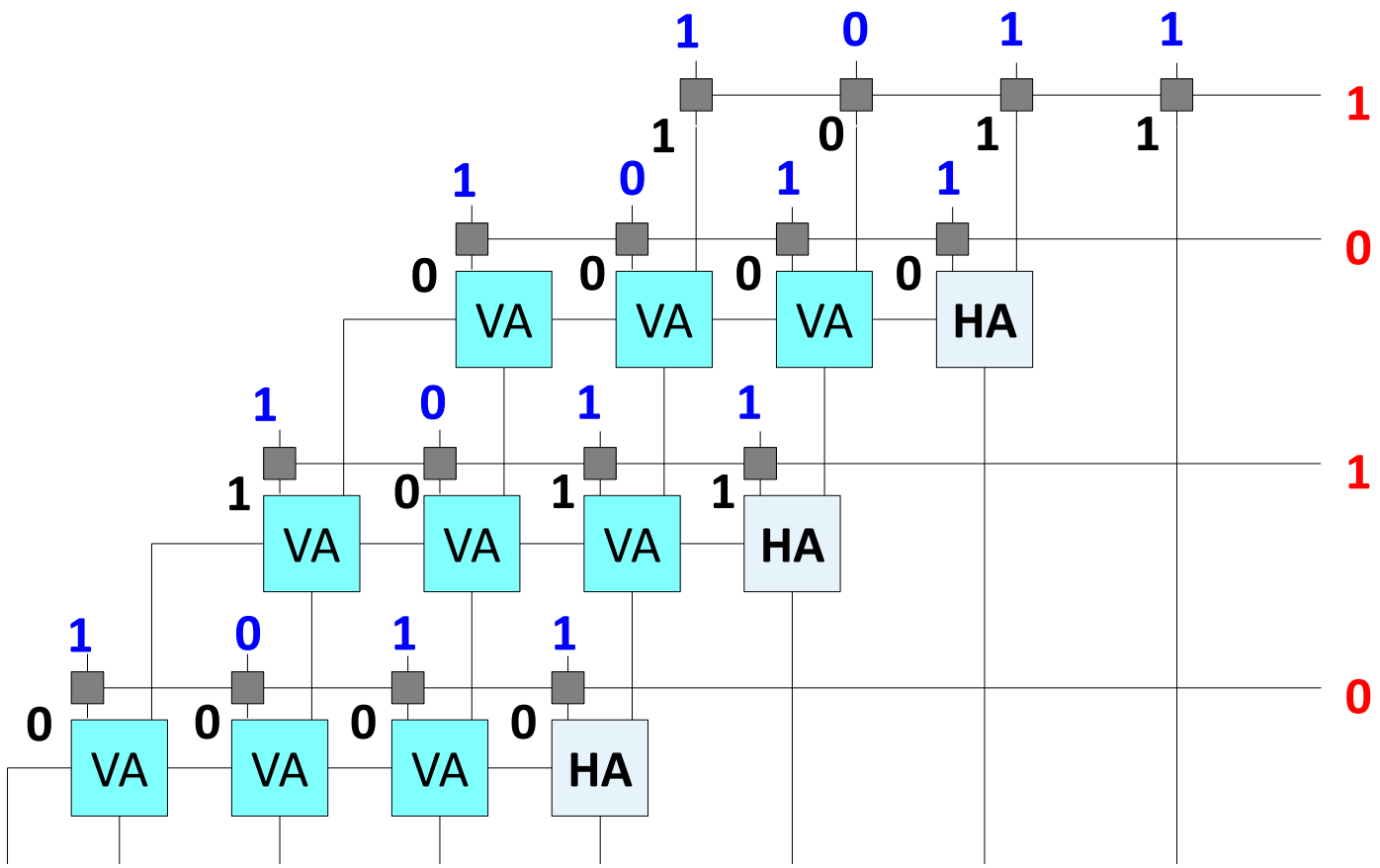
Paralleler Multiplizierer #2

Beispiel: 1011 * 0101

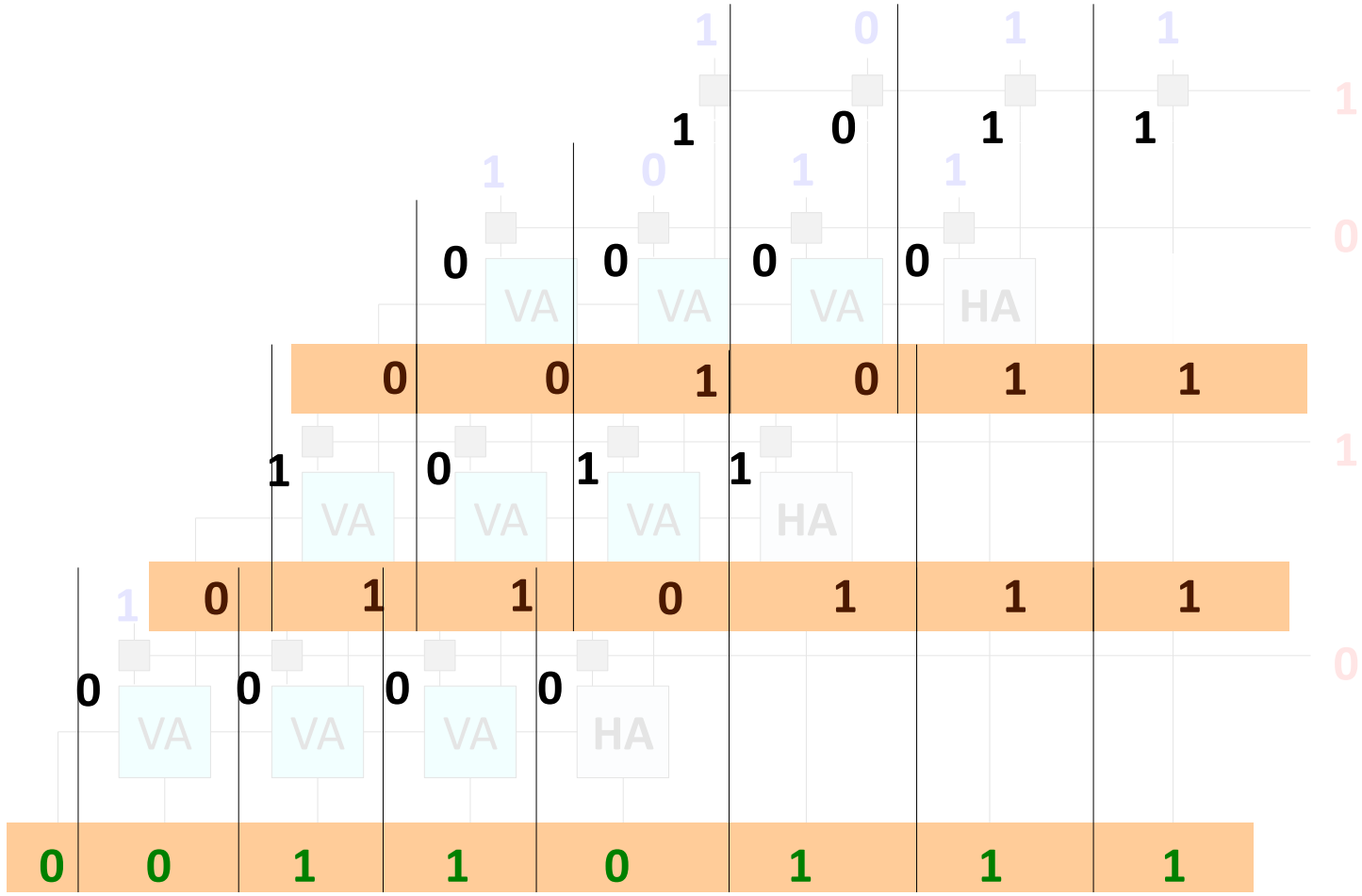


Paralleler Multiplizierer #3

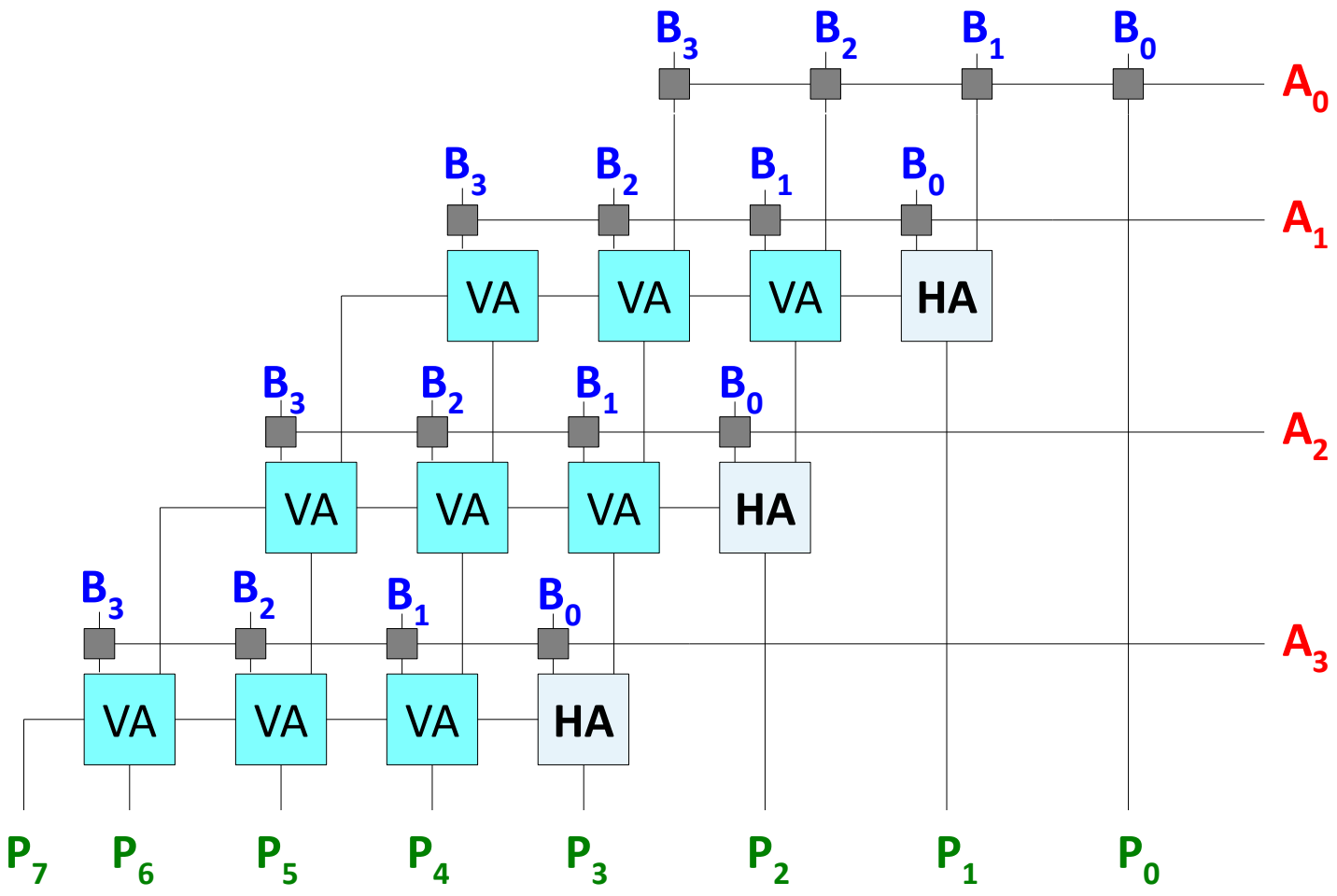
Beispiel: 1011 * 0101



Beispiel: 1011 * 0101

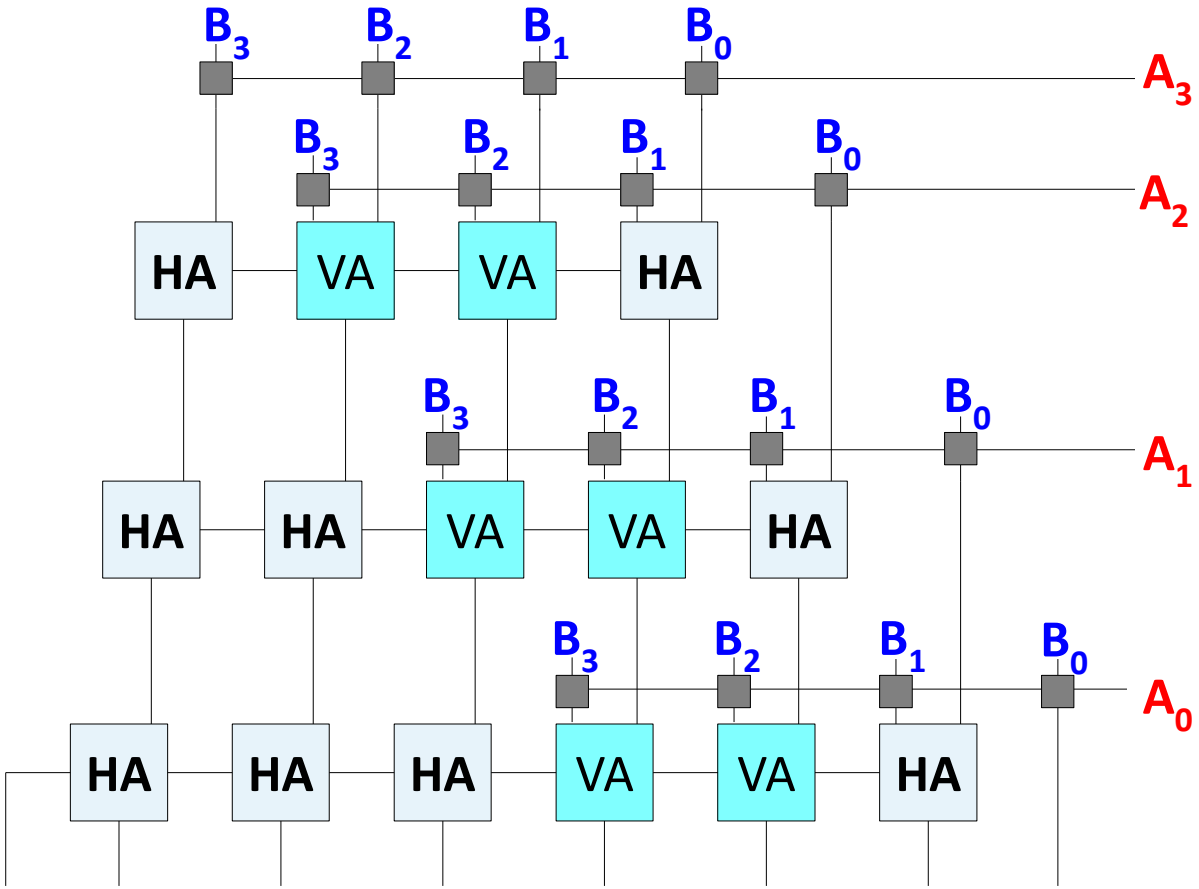


Paralleler Multiplizierer #1



Multiplizierer: Alternative Struktur

- Beginn bei Teilprodukt mit MSB von *A*
- Ungünstiger, da unregelmäßige Struktur



Multiplizierer mit Hilfe des Carry-Save Addierers

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Daraus entstehende Aufgabe:

Schnelle Addition von n Partialprodukten der Länge $2n$.

Mit CLAs lösbar mit Kosten $O(n^2)$,

Tiefe $O(n \log(n))$ bei *linearem Aufsummieren* der Partialprodukte $((\ldots(pp_0+pp_1)+pp_2)+\ldots)+pp_{n-1})$,

Tiefe $O(\log^2 n)$ bei *baumartigem Zusammenfassen* der Partialprodukte.

Verbesserung

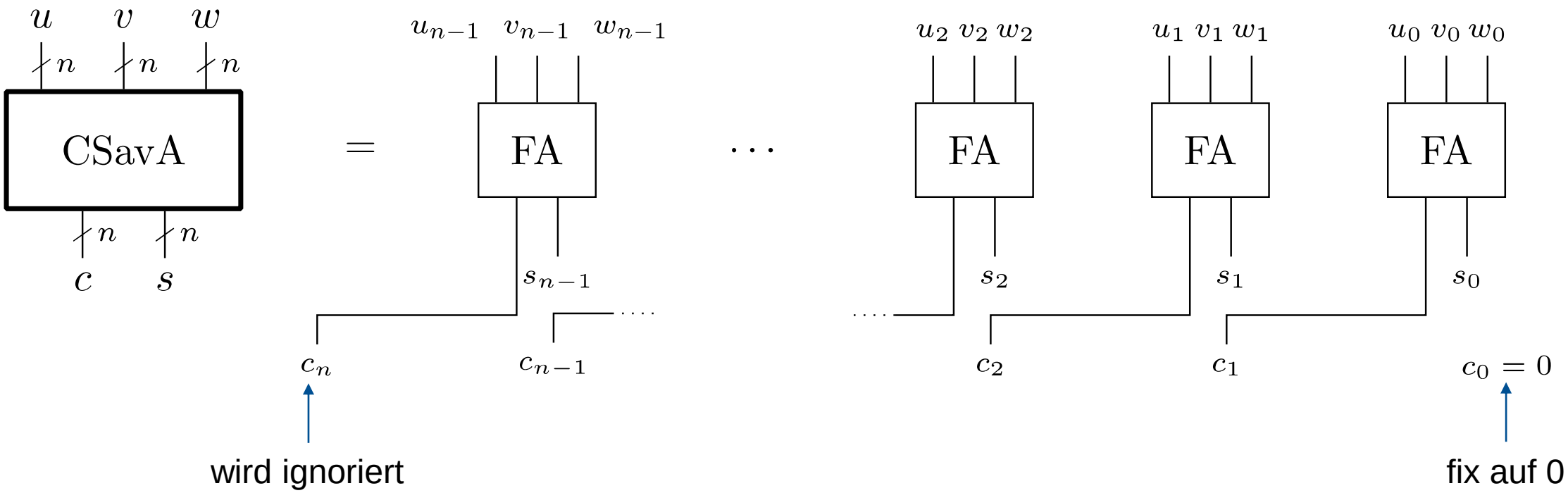
Verwende Carry-Save-Addierer.

Reduktion von **drei** Eingabeworten <u>, <v>, <w> zu
zwei Ausgabeworten <s>, <c> mit <u> + <v> + <w> = <s> + <c>

$$
\begin{array}{cccccccc}
 & & u_{n-1} & u_{n-2} & \cdots & u_2 & u_1 & u_0 \\
+ & & v_{n-1} & v_{n-2} & \cdots & v_2 & v_1 & v_0 \\
+ & & w_{n-1} & w_{n-2} & \cdots & w_2 & w_1 & w_0 \\
\hline
 & c_n & c_{n-1} & c_{n-2} & \cdots & c_2 & c_1 & c_0 \\
 & 0 & s_{n-1} & s_{n-2} & \cdots & s_2 & s_1 & s_0
\end{array}
$$

Gelöst durch Nebeneinandersetzen von Volladdierern
(keine Carry-Chain!).

Carry-Save Addierer für Partialprodukte



Bemerkung zum Aufbau des CSavA

Speziell bei Partialprodukten:

Reduziere drei $2n$-Bit-Zahlen zu zwei $2n$-Bit-Zahlen.

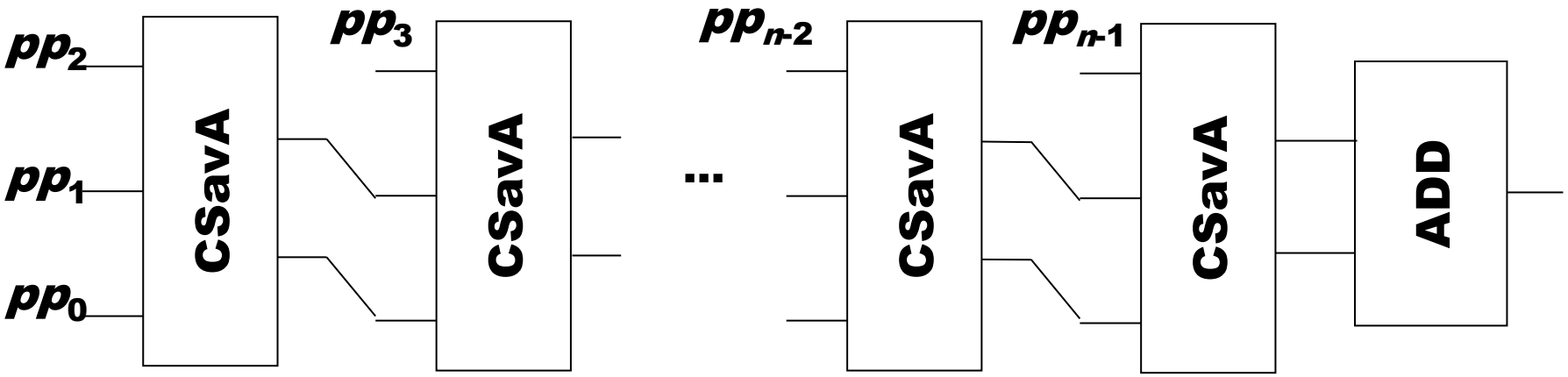
$$
\begin{array}{cccccccc}
 & pp^0_{2n-1} & pp^0_{2n-2} & \cdots & pp^0_2 & pp^0_1 & pp^0_0 \\
+ & pp^1_{2n-1} & pp^1_{2n-2} & \cdots & pp^1_2 & pp^1_1 & pp^1_0 \\
+ & pp^2_{2n-1} & pp^2_{2n-2} & \cdots & pp^2_2 & pp^2_1 & pp^2_0 \\
\hline
 & c_{2n-1} & c_{2n-2} & \cdots & c_2 & c_1 & 0 \\
 & s_{2n-1} & s_{2n-2} & \cdots & s_2 & s_1 & s_0
\end{array}
$$

($c_{2n} = 0 \Rightarrow$ Carry-Ausgang des letzten FA nicht verwendet)

1. Serielle Lösung:

- Hintereinanderschalten von *n*-2 CSavA-Addierern der Länge 2*n*
  → Fasse *n* Partialprodukte zu zwei
  2*n*-Bit-Worten zusammen
- Addiere die 2*n*-Bit-Worte mit CLA
- *siehe Abb. der Addierstufe*
- **Kosten** $O(n^2)$, **Tiefe** $O(n)$

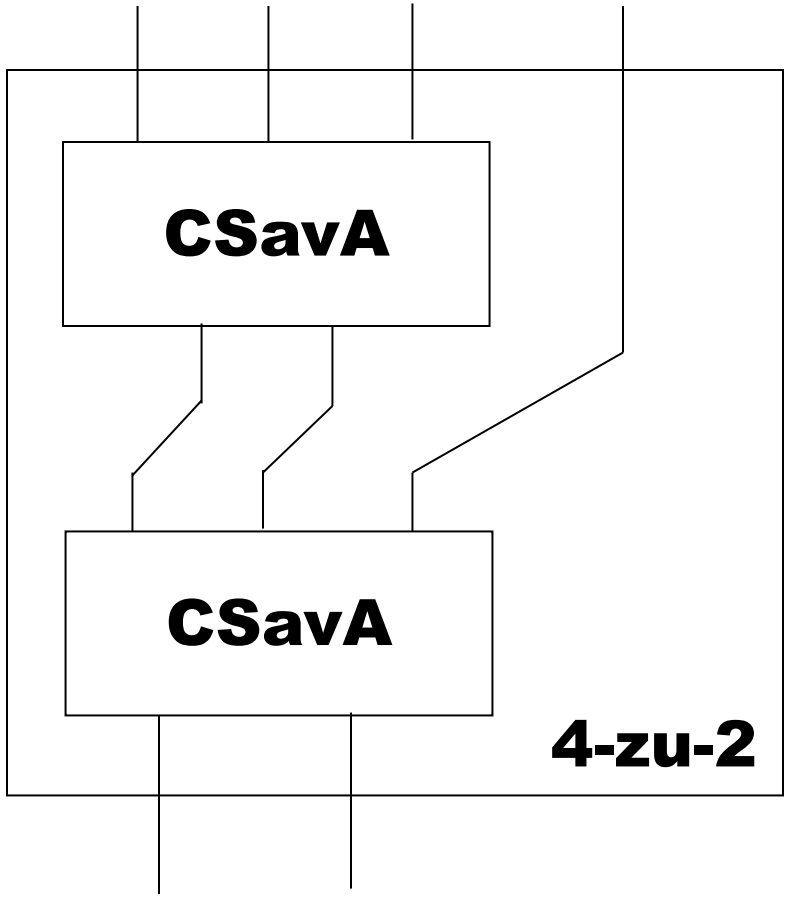
Addierstufe im Multiplizierer



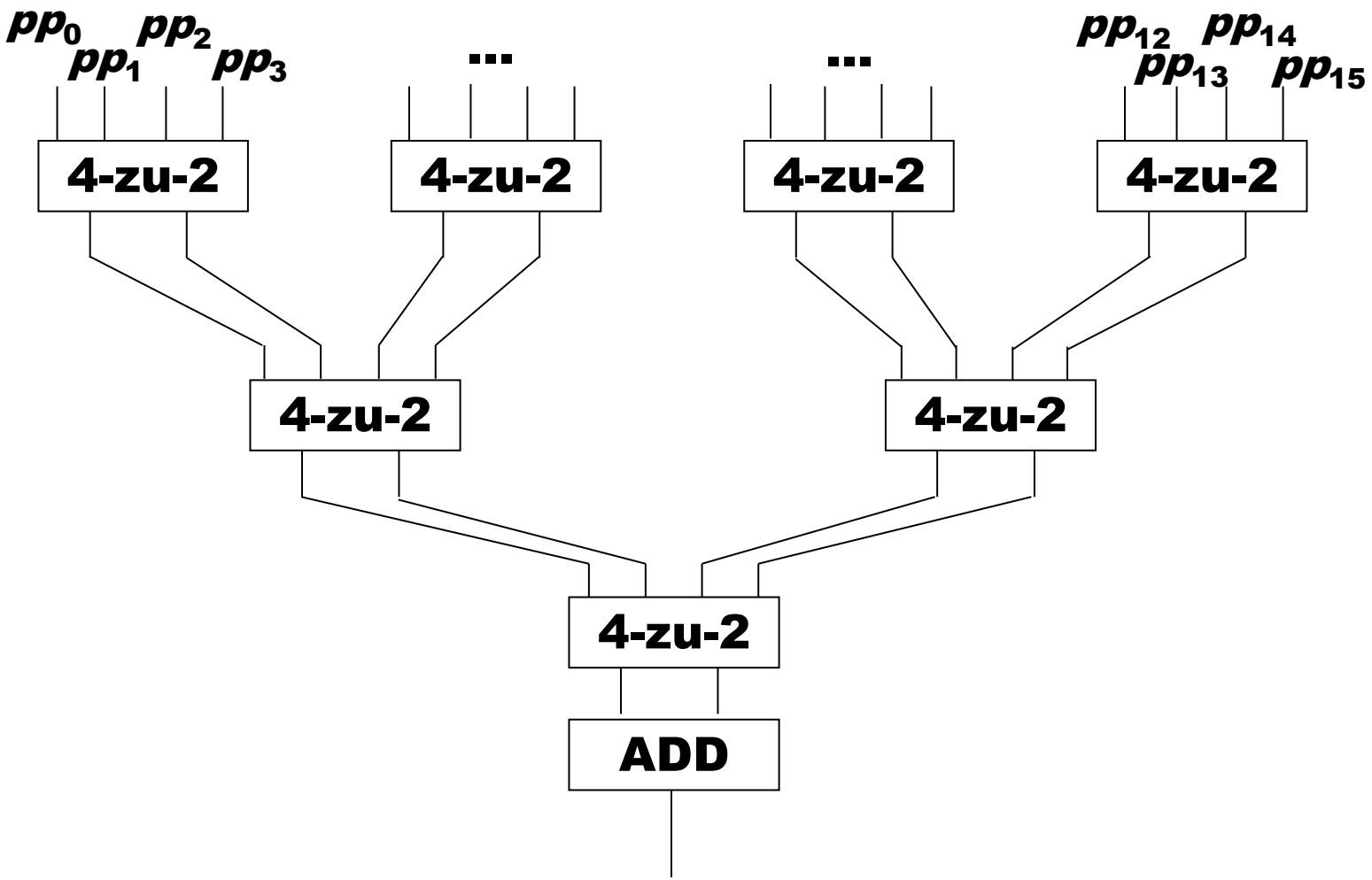
2. Baumartige Lösung:

- Neue Grundzelle zur Reduktion von vier 2*n*-Bit Eingabeworten zu zwei Ausgabeworten, bestehend aus zwei CSavAs (*siehe Abb. zur Reduktionszelle*)
- Baumartiges Zusammenfassen der Partialprodukte mit 4-zu-2-Bausteinen zu zwei 2*n*-Bit-Worten
- Addiere die 2*n*-Bit-Worte mit CLA
- *siehe Abb. der Addierstufe mit log. Zeit*
- **Kosten** $O(n^2)$, **Tiefe** $O(\log n)$

4-zu-2 Reduktions-Grundzelle



Addierstufe des log-Zeit-Multiplizierers für 16 Bit



Multiplexer

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Multiplexer

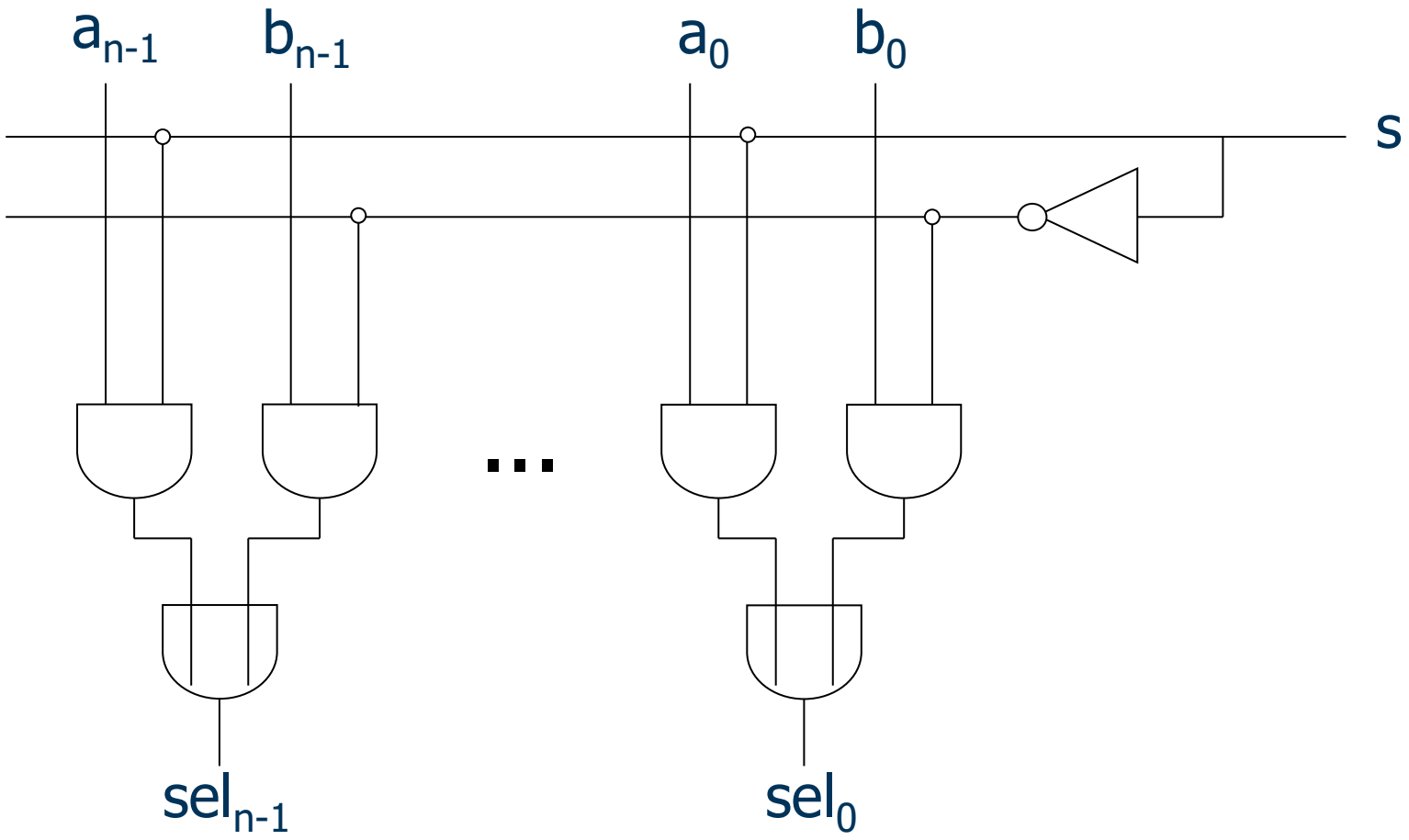
Ein n**-Bit-Multiplexer** (MUX_n) ist ein Schaltkreis, der die folgende Funktion berechnet:

$$sel_n : \mathbf{B}^{2n+1} \rightarrow \mathbf{B}^n \quad \text{mit}$$

$$\mathrm{sel}_n(a_{n-1},\ldots,a_0,b_{n-1},\ldots,b_0,s) = \begin{cases} (a_{n-1}\ldots a_0), \text{falls } s = 1 \\ (b_{n-1}\ldots b_0), \text{falls } s = 0 \end{cases}$$

$$(\mathrm{sel}_n)_i = s \cdot a_i + \bar{s} \cdot b_i$$

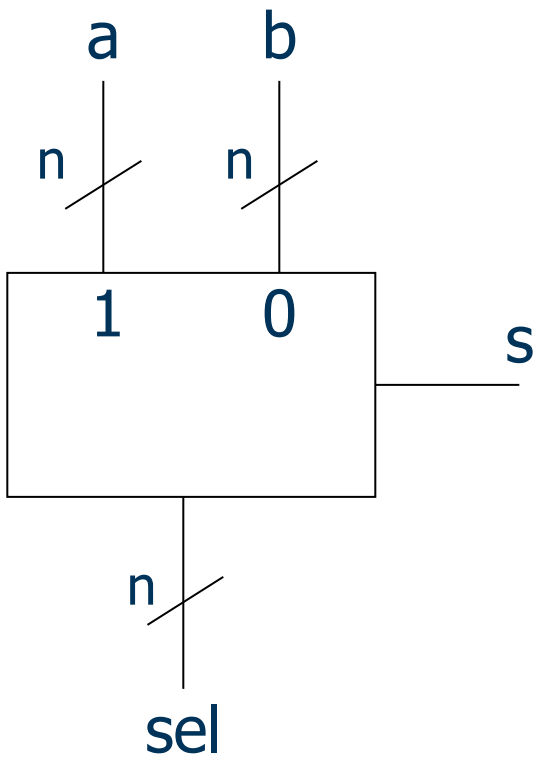
Realisierung eines n-Bit Multiplexer



Kosten eines n-Bit MUX

Gatter: $3n + 1$

Tiefe: 3



ALU

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

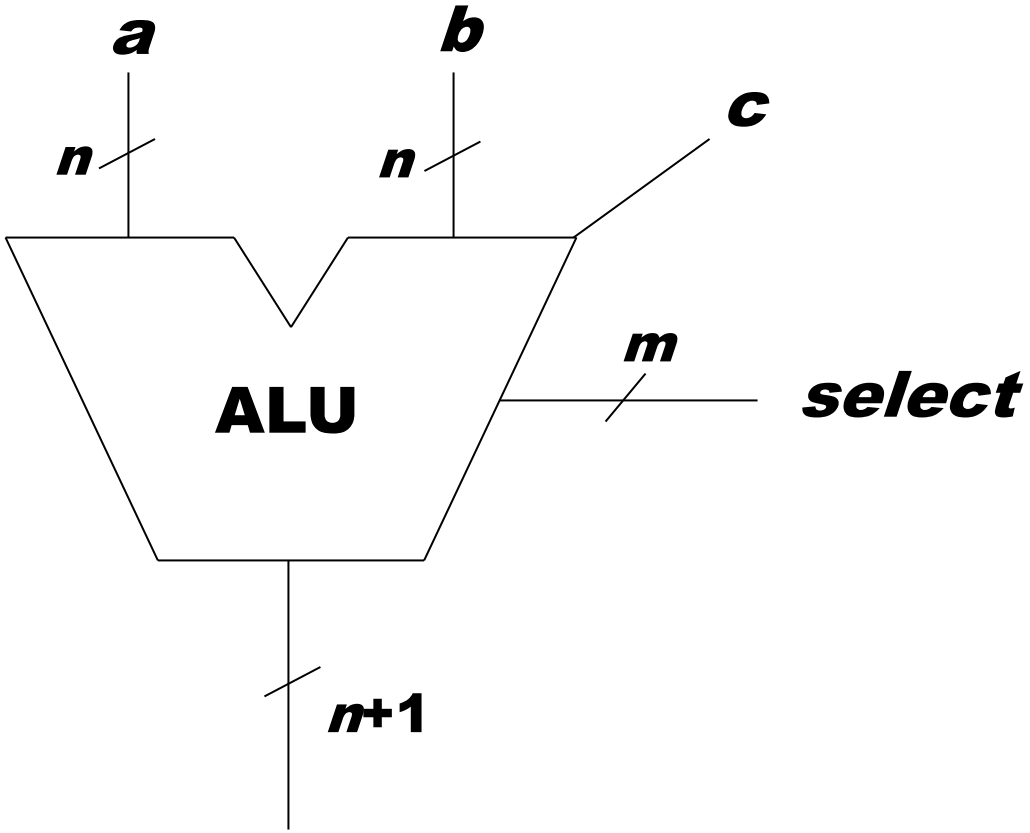
Aufbau einer ALU

ALU = **Arithmetic Logic Unit** zur Berechnung von arithmetischen und logischen Basisoperationen

n-Bit-ALU mit:

- zwei *n*-Bit-Operanden *a*, *b*, Eingangscarry *c*
- *m*-Bit *select*-Eingang, der auswählt, welche Funktion ausgeführt wird
- (*n*+1)-Bit-Ausgang

Schaltzeichen einer *n*-Bit-ALU



Mögliche Realisierungen einer ALU

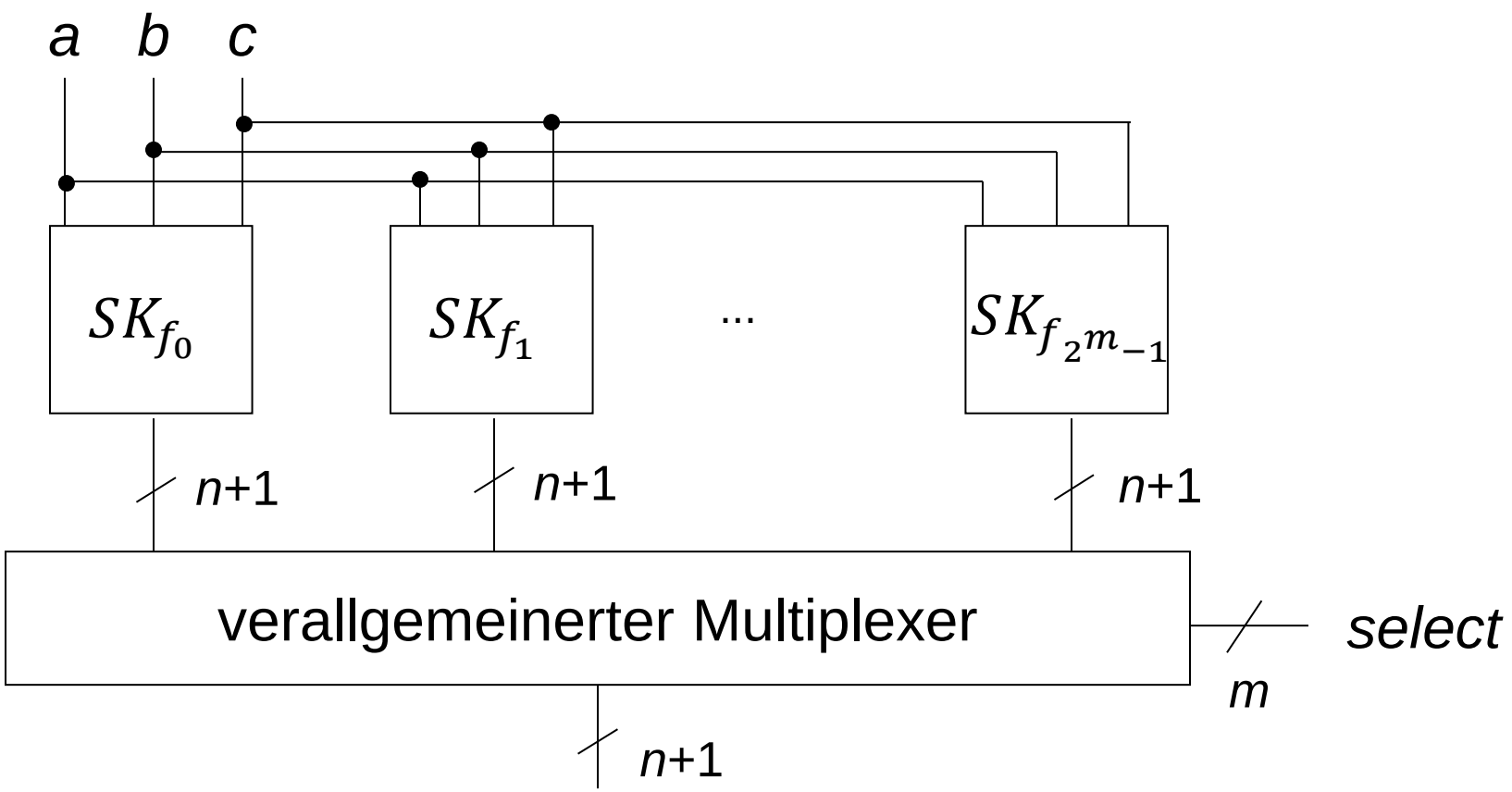
1. *Möglichkeit:*

   Realisiere Fkt. $f_0, \ldots, f_{2^m-1}$ getrennt durch SK_{f_i}

   für f_i, dann Auswahl durch verallgemeinerten Multiplexer

   (*siehe Abb. der möglichen Realisierung*)

2. *Möglichkeit:*

   gemeinsame Behandlung ähnlicher Funktionen

Realisierung der *n*-Bit-ALU (Einzelfunktionen getrennt)